

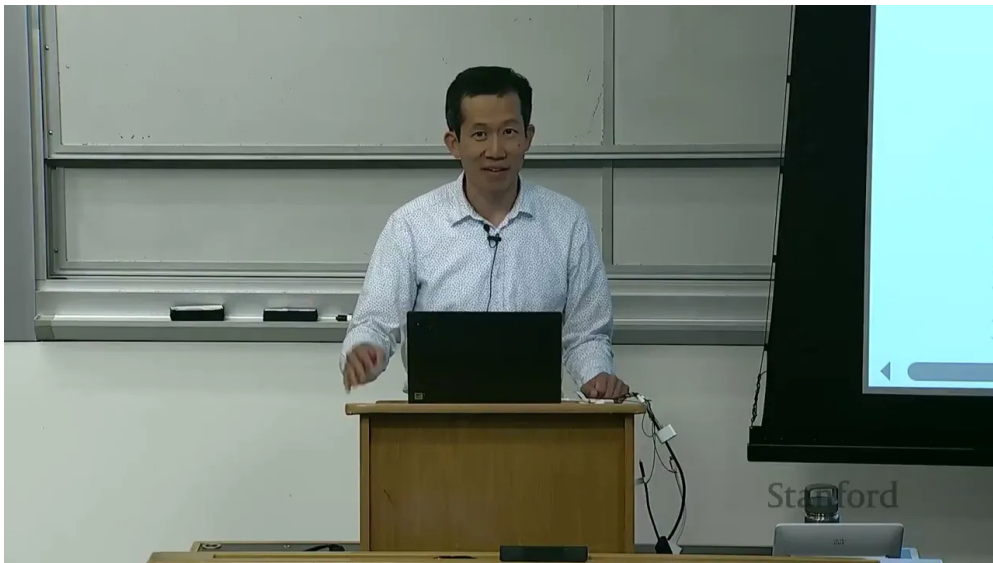
课程笔记

CS336 Lecture 17: 对齐——强化学习（下）

Stanford CS336: Language Modeling from Scratch (Spring 2025)

讲师: Percy Liang 笔记: ZCode

2026-07-14



视频作者/频道: Stanford CS336

发布日期: 2025 Spring

视频时长: 76 分钟

视频链接: <https://www.youtube.com/watch?v=JdGFdVia0Jk>

Contents

1 引言：本讲的定位	2
2 语言模型 RL 的形式化设置	2
2.1 状态、动作、奖励	2
2.2 转移动力学与“可规划性”	4
2.3 策略与目标函数	5
3 策略梯度：从目标到更新	6
3.1 策略梯度定理	6
3.2 朴素策略梯度及其 SFT 类比	7
3.3 高方差与稀疏奖励陷阱	8
4 方差、基线与优势函数	9
4.1 两状态玩具例子：理解高方差	9
4.2 基线的思想	10
4.3 基线如何降方差：回到例子	11
4.4 最优基线与启发式选择	12
4.5 优势函数	13
4.6 统一的 delta 形式	14
5 GRPO：动机与伪代码	15
5.1 为什么 GRPO 在 2024 年才出现	15
5.2 GRPO 伪代码全貌	16
6 玩具任务：排序与奖励设计	17
6.1 为什么用排序任务	17
6.2 两种奖励函数与部分信用	18
7 极简模型与推理	19
7.1 模型架构	19
7.2 前向与采样	20
8 Delta、Log-prob 与损失	21
8.1 计算 delta：算法菜单	21
8.2 计算 log-prob	22
8.3 no_grad 插曲：冻结的艺术	23
8.4 GRPO 的 clipped loss	24
8.5 KL 惩罚：低方差估计	25
9 完整训练循环	26
9.1 三层循环结构	26
9.2 参考模型与“慢节奏”	27
10 Assignment 5 与延伸	27
10.1 与作业的对应	27
10.2 过程奖励与多轮交互	28
10.3 本讲小结	28
10.4 配套阅读	28

1 导言：本讲的定位

这是课程的最后一堂主讲课。Percy Liang 开篇就定位了本讲的性质：上一堂课 Tatsu 给出了“带验证器奖励的强化学习”（RL from verifiable rewards）的总览，介绍了 PPO、GRPO 等策略梯度算法的概念。本讲不引入新内容，而是对策略梯度（特别是 GRPO）的机制做深度拆解——把抽象的算法落到代码与数学上，并配套一个可以在笔记本上跑通的玩具任务。

本讲三条主线

1. 建立形式化：把语言模型的 RL 设置（状态、动作、奖励、转移动力学）讲精确。
2. 方差与基线：为什么朴素策略梯度噪声大，基线（baseline）/优势函数（advantage）如何降低方差。
3. GRPO 全流程代码：用一个“排序”玩具任务和极简模型，逐块讲清 rollout、reward、delta、log-prob、clipped loss、KL 惩罚与训练循环。

Percy 特别提醒：GRPO 的具体形式不是最重要的——明年可能就有 GRPO2，但策略梯度的核心数学是稳定的，“明年我还会讲同样的内容”。这一讲的价值在于把“算法菜单”和“取舍直觉”交到学生手里。

2 语言模型 RL 的形式化设置

2.1 状态、动作、奖励

强化学习的三大要素在语言模型里有非常具体的含义。Percy 强调要把“显而易见”的东西讲清楚。

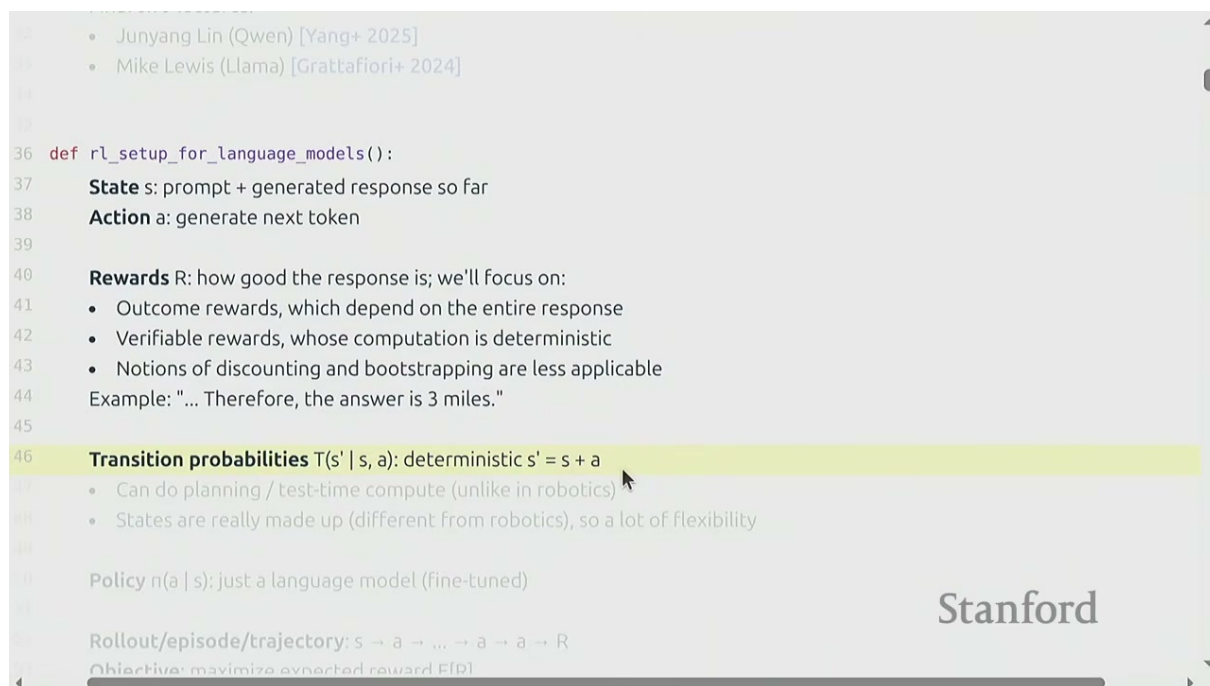


Figure 1: 语言模型 RL 的设置：状态 = prompt + 已生成响应；动作 = 下一个 token；奖励 = 响应的好坏¹

要素	在语言模型中的含义
状态 s	prompt 加上到目前为止生成的响应（即已写出的 token 序列）
动作 a	生成下一个 token
策略 $\pi_{\theta}(a s)$	语言模型本身，给定上文输出下一 token 分布
奖励 r	整个响应有多好；本课聚焦结果奖励（outcome reward）

本课的两个核心假设：

¹视频画面时间区间：00:02:45 – 00:03:30。

- 结果奖励：奖励是整个响应的函数，不是中间每一步都有奖励。
- 可验证奖励：奖励计算是确定性的——某个函数直接返回数值（如对答案），而不需要问人。这与 RLHF 中需要从偏好数据估计的奖励模型不同。

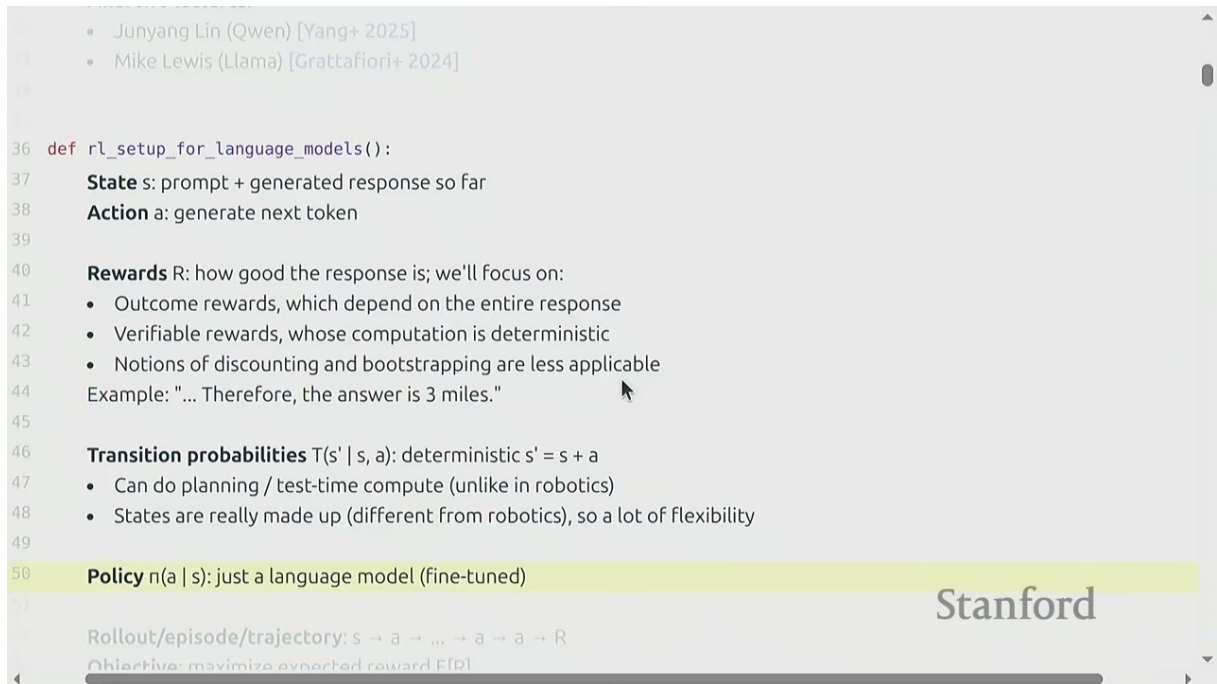


Figure 2: 结果奖励的典型形态：数学题里模型生成一段思维链，最后输出“答案是 3 英里”，奖励函数提取答案并与 ground truth 比对²

稀疏与延迟奖励

整条轨迹只在最后得到一个奖励信号。这带来了稀疏（sparse）与延迟（delayed）两大难题：如果策略太差、rollout 全错，梯度近似为零，模型就“卡住”了。但也正因为如此，概念上更简单——不必处理折扣（discounting）和 bootstrapping。

²视频画面时间区间：00:04:30 – 00:05:15。

2.2 转移动力学与“可规划性”

Transition probabilities $p(s | s', a)$: deterministic $s' = s + a$

- Can do planning / test-time compute (unlike in robotics)
- States are really made up (different from robotics), so a lot of flexibility

Policy $n(a | s)$: just a language model (fine-tuned)

Rollout/episode/trajectory: $s \rightarrow a \rightarrow \dots \rightarrow a \rightarrow a \rightarrow R$

Objective: maximize expected reward $E[R]$
(where the expectation is taken over prompts s and response tokens a)

```
def policy_gradient():
```

For notational simplicity, let a denote the entire response.

We want to maximize expected reward with respect to the policy n :

$$E[R] = \int p(s) n(a | s) R(s, a)$$

Obvious thing to do is to take the gradient:

$$\nabla E[R] = \int p(s) \nabla n(a | s) R(s, a)$$
$$\nabla E[R] = \int p(s) n(a | s) \nabla \log n(a | s) R(s, a)$$
$$\nabla E[R] = E[\nabla \log n(a | s) R(s, a)]$$

Stanford

Figure 3: 转移动力学极简：新状态 = 旧状态拼接新 token；因为动力学已知，可以做规划（test-time compute）³

在语言模型设定里，转移动力学异常简单：

$$s_{t+1} = s_t \oplus a_t$$

也就是把新 token 直接拼到当前序列后面。这有两个深远含义：

为什么语言模型 RL 与机器人 RL 不同

1. 可以规划：因为你知道世界动力学，可以模拟未来、做 test-time compute（搜索/采样）。机器人学家做梦都想有这种能力。
2. 状态是“造出来”的：状态不是物理量（关节角、图像），而是模型自己生成的 token。因此模型可以自由构造自己的 scratchpad（思维链）来推导答案。难点不是“能否到达某状态”（写什么 token 都行），而是这些 token 是否真的导向正确答案。

这种“状态自由”是把双刃剑：它解锁了思维链与自我改进，但也让信用分配（credit assignment）更难——你不知道是早期决策重要还是结尾重要。

³视频画面时间区间：00:05:30 – 00:06:15。

2.3 策略与目标函数

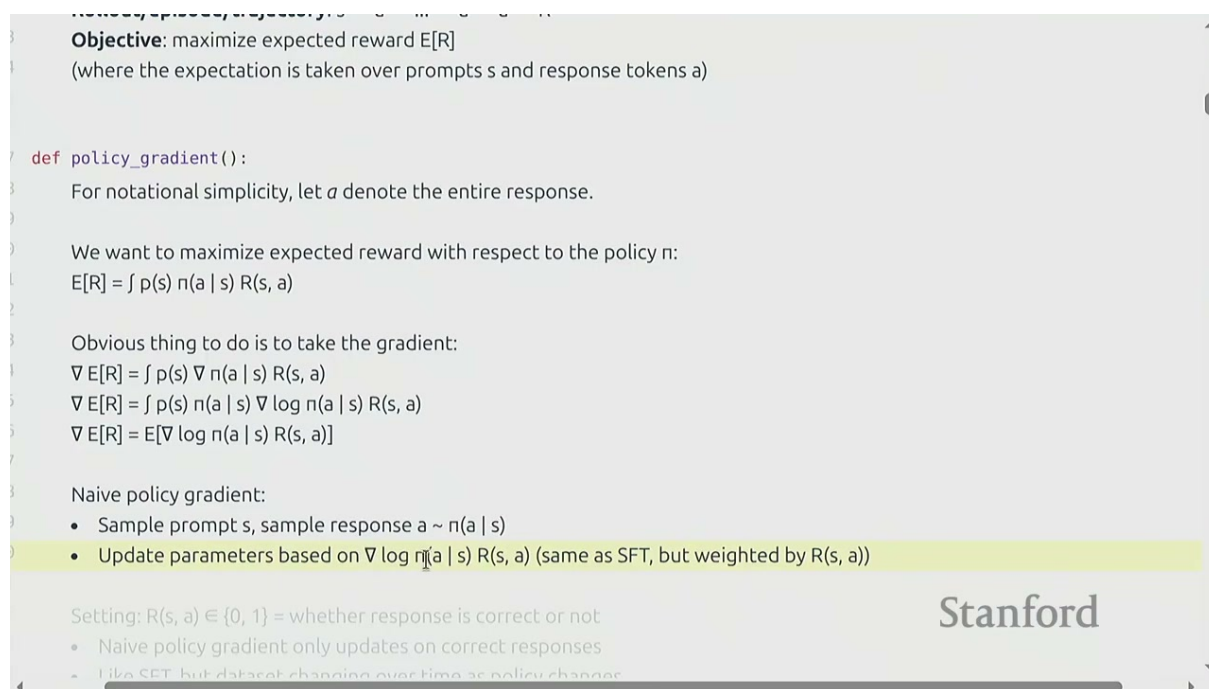


Figure 4: 策略即语言模型；目标是最大化期望奖励，期望同时遍历 prompt 分布与策略采样⁴

策略 π_θ 通常由一个预训练语言模型微调而来。整条从 prompt 到响应的过程也叫 rollout / episode / trajectory。形式化目标为：

$$J(\theta) = \mathbb{E}_{s \sim p(s), a \sim \pi_\theta(\cdot | s)} [r(s, a)]$$

- $p(s)$: 环境给出的 prompt 分布
- $\pi_\theta(\cdot | s)$: 当前策略对响应的采样分布
- $r(s, a)$: 结果奖励

为简化记号，后续用 a 直接表示整条响应（而非单个 token），因为结果奖励下这与“一次生成所有动作”等价。

⁴视频画面时间区间：00:07:45 – 00:08:30。

3 策略梯度：从目标到更新

3.1 策略梯度定理

def policy_gradient():

For notational simplicity, let a denote the entire response.

We want to maximize expected reward with respect to the policy π :

$$E[R] = \int p(s) \pi(a | s) R(s, a)$$

Obvious thing to do is to take the gradient:

$$\nabla E[R] = \int p(s) \nabla \pi(a | s) R(s, a)$$
$$\nabla E[R] = \int p(s) \pi(a | s) \nabla \log \pi(a | s) R(s, a)$$
$$\nabla E[R] = E[\nabla \log \pi(a | s) R(s, a)]$$

Naive policy gradient:

- Sample prompt s , sample response $a \sim \pi(a | s)$
- Update parameters based on $\nabla \log \pi(a | s) R(s, a)$ (same as SFT, but weighted by $R(s, a)$)

Setting: $R(s, a) \in \{0, 1\}$ = whether response is correct or not

- Naive policy gradient only updates on correct responses
- Like SFT, but dataset changing over time as policy changes

Challenge: high noise/variance

In this setting, sparse rewards (few responses get reward 1, most get 0)

In contrast: in RLHF, reward models (learned from pairwise preferences) are more continuous

Stanford

Figure 5: 策略梯度推导：对期望奖励求梯度，梯度只作用于策略；用 $\nabla \log$ 的链式法则改写为可采样估计⁵

策略梯度的核心是把“对期望求导”转化为“可采样的估计”。对 $J(\theta) = \int p(s) \pi_\theta(a | s) r(s, a) ds da$ 求梯度，梯度只触碰策略项：

$$\nabla_\theta J(\theta) = \mathbb{E}_{s,a} [r(s, a) \nabla_\theta \log \pi_\theta(a | s)]$$

为什么能这样换？

关键恒等式 $\nabla \log \pi = \nabla \pi / \pi$ （即 $\nabla \pi = \pi \cdot \nabla \log \pi$ ）。把它代回积分，多出的 π 把期望“还原”回来。这就是“策略梯度定理”，本质就是对数导数技巧（log-derivative trick）。

⁵视频画面时间区间：00:09:00 – 00:09:45。

3.2 朴素策略梯度及其 SFT 类比

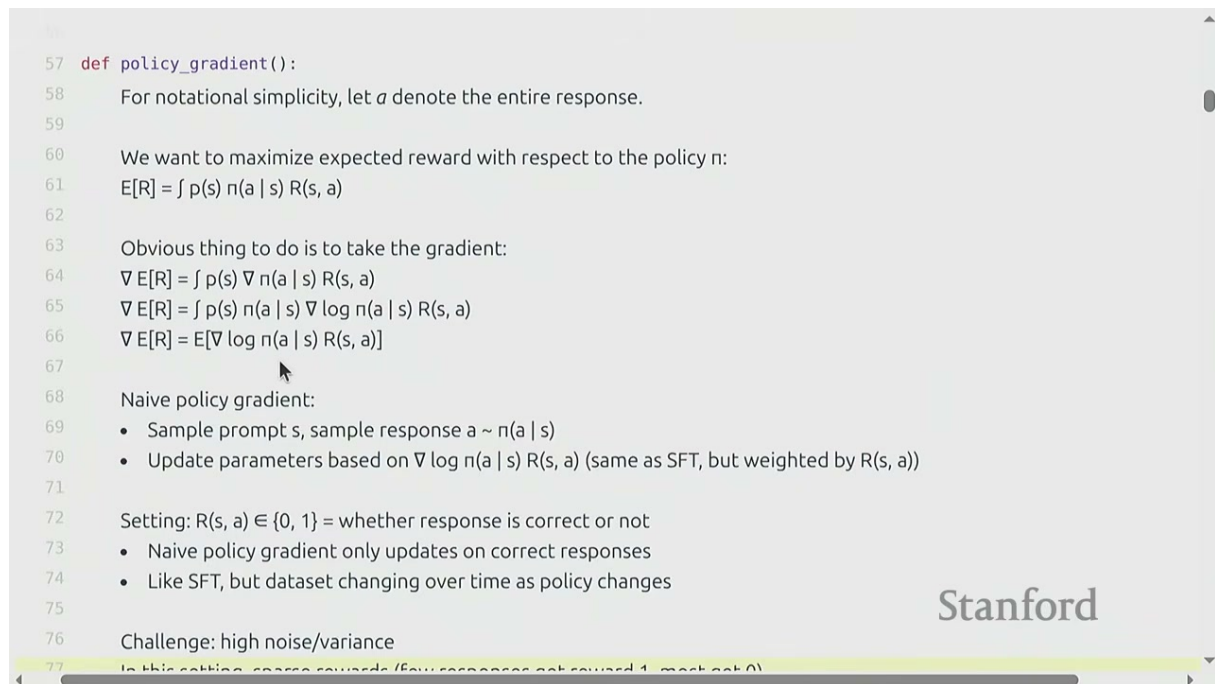


Figure 6: 朴素策略梯度 = 带 reward 加权的 SFT：采样一个 prompt、采样一个响应、按 $r \cdot \nabla \log \pi$ 更新⁶

朴素策略梯度就是 SGD 的直接套用：

$$\theta \leftarrow \theta + \eta \cdot r(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s)$$

与 SFT 的深刻类比

SFT 是人写出 a ，模型模仿 a 、最大化其概率。朴素策略梯度里 a 由模型自己生成，但整体被 reward 加权。若 reward 是 $\{0, 1\}$ ：

- $r = 1$ （答对）：等价于在该响应上做 SFT，提升概率。
- $r = 0$ （答错）：梯度为零，完全忽略。

唯一区别是数据集随时间变化：每次更新后策略变了，重新采样的数据集分布也变了（且希望越来越好）。

⁶视频画面时间区间：00:10:45 – 00:11:30。

3.3 高方差与稀疏奖励陷阱

60 We want to maximize expected reward with respect to the policy π :

61 $E[R] = \int p(s) \pi(a | s) R(s, a)$

62

63 Obvious thing to do is to take the gradient:

64 $\nabla E[R] = \int p(s) \nabla \pi(a | s) R(s, a)$

65 $\nabla E[R] = \int p(s) \pi(a | s) \nabla \log \pi(a | s) R(s, a)$

66 $\nabla E[R] = E[\nabla \log \pi(a | s) R(s, a)]$

67

68 Naive policy gradient:

69 • Sample prompt s , sample response $a \sim \pi(a | s)$

70 • Update parameters based on $\nabla \log \pi(a | s) R(s, a)$ (same as SFT, but weighted by $R(s, a)$)

71

72 Setting: $R(s, a) \in \{0, 1\}$ = whether response is correct or not

73 • Naive policy gradient only updates on correct responses

74 • Like SFT, but dataset changing over time as policy changes

75

76 Challenge: high noise/variance

77 In this setting, sparse rewards (few responses get reward 1, most get 0)

78 In contrast: in RLHF, reward models (learned from pairwise preferences) are more continuous

Stanford

Baselines

Figure 7: 稀疏奖励陷阱：策略太差时 rollout 全错， $r = 0$ ，梯度为零，模型卡死；与 RLHF 的连续 reward 形成对比⁷

监督学习的 SGD 也会有噪声，但 RL 是“另一个量级”的噪声。在 $\{0, 1\}$ 奖励的稀疏设定下：

卡死问题

如果策略太差，几乎所有 rollout 都得 $r = 0$ ，梯度近似为零，参数不动。监督学习至少保证每步都朝某方向更新，而 RL 不保证。这正是为什么需要基线/奖励塑形/partial credit 等技巧。唯一出路：只要偶尔能蒙对几道简单题拿到 reward，就能更新、泛化，下一轮采样到略难的题也能做对——数据集 reward 单调上升。

Percy 回答了一个好问题：“为什么不把 0 改成 -1？”——他指出这会被基线机制自动处理：用 reward 减去均值后，原来全 0 的 batch 也会出现负的 advantage，从而对错答施加“推远”的更新。

⁷ 视频画面时间区间：00:12:15 – 00:13:00。

4 方差、基线与优势函数

4.1 两状态玩具例子：理解高方差

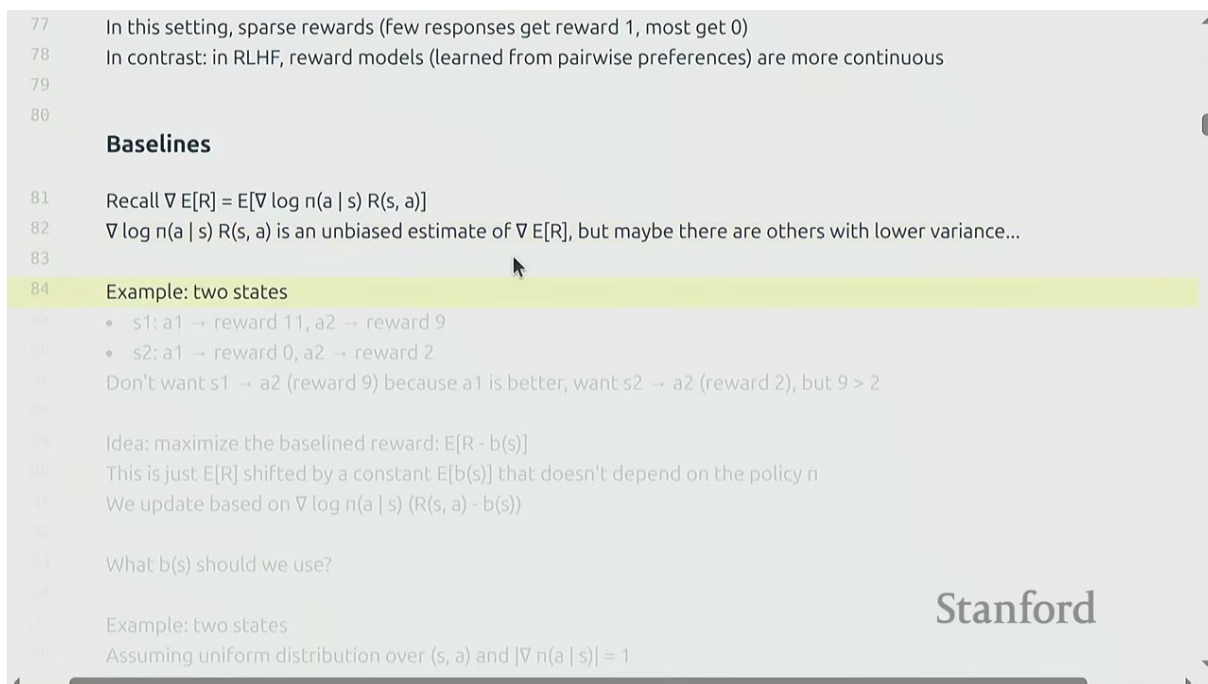


Figure 8: 两状态例子： S_1 （简单题）的次优动作也拿 reward 9，比 S_2 （难题）的最优动作（reward 2）还高，朴素梯度会被误导⁸

Percy 给了一个具象例子来展示“高方差”到底是什么。设两状态 S_1 （易）、 S_2 （难），各有两动作：

状态	动作 A1 reward	动作 A2 reward	最优策略
S_1 （易题）	11	9	选 A1
S_2 （难题）	0	2	选 A2

被误导的局部信号

朴素梯度只看绝对 reward。若在 S_1 上选了次优 A2 拿到 9，会得到“很大的正向梯度”；而在 S_2 上选最优 A2 只拿 2，梯度小。于是模型拼命强化 $S_1 \rightarrow A2$ ，把概率质量从真正的最优 A1 上挤走，陷入局部最优。整体期望上数学终究是对的，但需要“看到全貌”才成立——单步采样的噪声正是源于此。

⁸视频画面时间区间：00:16:15 – 00:17:00。

4.2 基线的思想

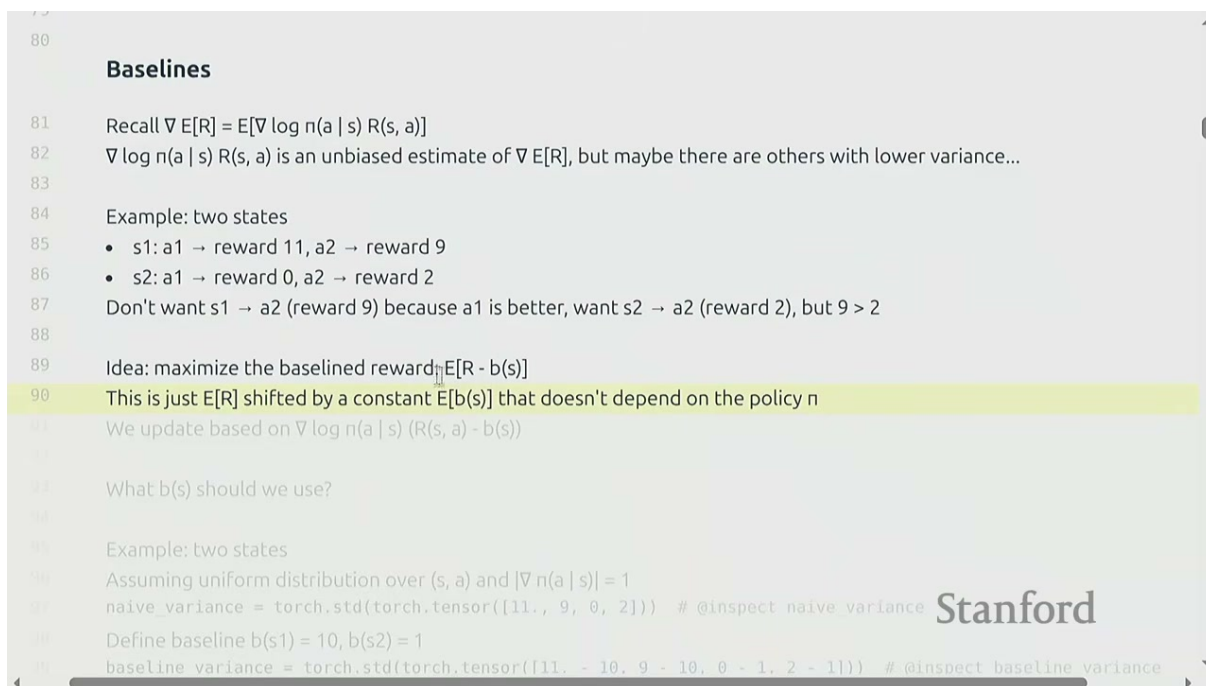


Figure 9: 基线思想：从 reward 减去一个只依赖 s 的函数 $B(s)$ ，不改变优化问题（期望为常数），但能降方差⁹

核心技巧：用 $r - B(s)$ 替换 r ，其中 B 只依赖状态、不依赖动作。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s,a} [(r(s, a) - B(s)) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

为什么减基线不改变梯度？

因为 $\mathbb{E}_{a \sim \pi(\cdot | s)} [\nabla_{\theta} \log \pi_{\theta}(a | s)] = \int \pi \cdot \frac{\nabla \pi}{\pi} da = \nabla_{\theta} \int \pi da = \nabla_{\theta} 1 = 0$ 。所以 $B(s)$ 项的期望贡献为零，梯度不变，只是估计的方差变了。这对任何不依赖 a 的 $B(s)$ 都成立。

⁹视频画面时间区间：00:19:30 – 00:20:15。

4.3 基线如何降方差：回到例子

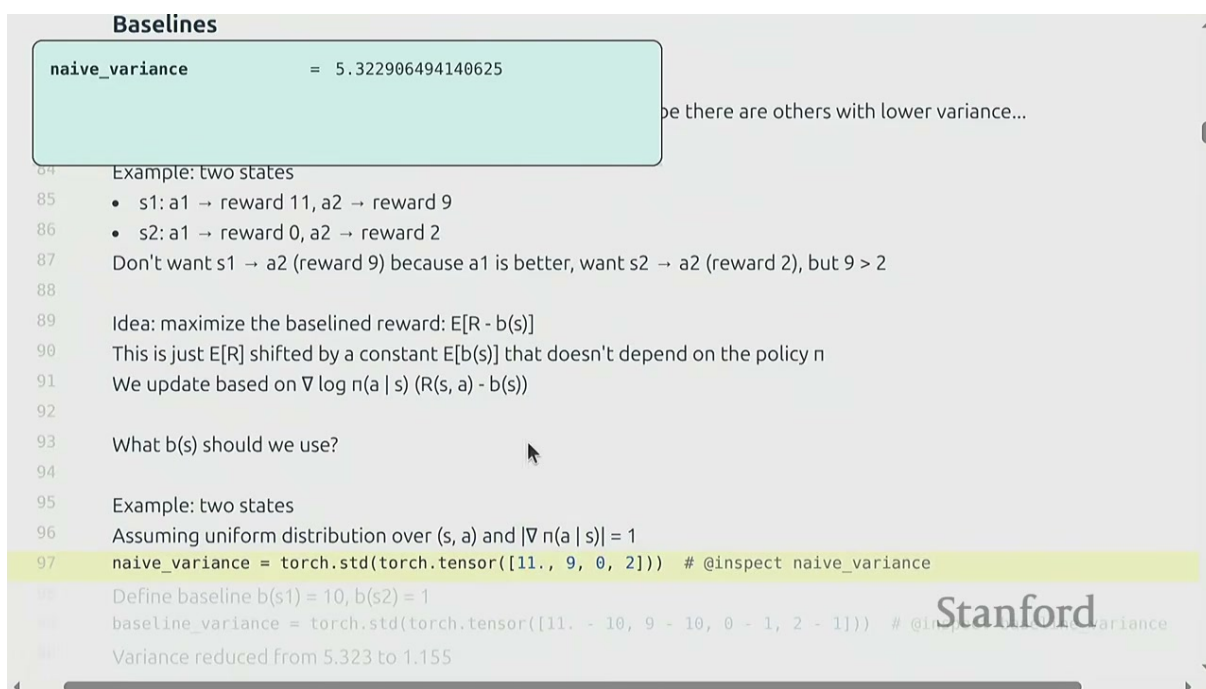


Figure 10: 同一例子加基线后：方差从 5.3 降到 1.1；降低方差意味着更快收敛¹⁰

回到两状态例子。设初始策略均匀、梯度模长为 1。原始 reward 取值 $\{11, 9, 0, 2\}$ ，方差为 5.3。取基线 $B(S_1) = 10$ 、 $B(S_2) = 1$ （即各状态的“平均 reward”），则 advantage 为：

$$S_1 : 11 - 10 = 1, 9 - 10 = -1; \quad S_2 : 0 - 1 = -1, 2 - 1 = 1$$

- 现在每个状态的“次优动作” advantage 为负，会被推远；最优动作 advantage 为正，会被拉近。
- 方差降到 1.1。降方差直接意味着更快收敛。

$B(s)$ 与“冻结策略”的关系

$B(s)$ 不能依赖当前可训练策略 π_θ 。但你可以冻结某个旧策略当常数用——只要不对其求导。实操中通常保留一个 old/reference 副本，更新若干步后再同步。PPO 用 value function 充当这个角色。

¹⁰视频画面时间区间：00:21:15 – 00:22:00。

4.4 最优基线与启发式选择

The screenshot shows a video player with a light blue background. A code editor window is open, displaying Python code. The code defines two variables: `naive_variance` with a value of 5.322906494140625 and `baseline_variance` with a value of 1.154700517654419. Below the code, there is a list of text lines. Line 89 says 'Idea: maximize the baselined reward: $E[R - b(s)]$ '. Line 90 says 'This is just $E[R]$ shifted by a constant $E[b(s)]$ that doesn't depend on the policy π '. Line 91 says 'We update based on $\nabla \log \pi(a | s) (R(s, a) - b(s))$ '. Line 92 is empty. Line 93 says 'What $b(s)$ should we use?'. Line 94 is empty. Line 95 says 'Example: two states'. Line 96 says 'Assuming uniform distribution over (s, a) and $|\nabla \log \pi(a | s)| = 1$ '. Line 97 shows the code `naive_variance = torch.std(torch.tensor([11., 9, 0, 2]))` with a comment `# @inspect naive_variance`. Line 98 says 'Define baseline $b(s_1) = 10, b(s_2) = 1$ '. Line 99 shows the code `baseline_variance = torch.std(torch.tensor([11. - 10, 9 - 10, 0 - 1, 2 - 1]))` with a comment `# @inspect baseline_variance`. Line 100 is highlighted in yellow and says 'Variance reduced from 5.323 to 1.155'. Line 101 is empty. Line 102 says 'Optimal $b^*(s) = E[(\nabla \log \pi(a | s))^2 R | s] / E[(\nabla \log \pi(a | s))^2 | s]$ (for one-parameter models)'. Line 103 says 'This is difficult to compute...'. Line 104 says '...so heuristic is to use the mean reward:'. Line 105 shows the code `b(s) = E[R | s]`. The Stanford logo is visible in the bottom right corner of the video player.

Figure 11: 最优基线有闭式解但难以计算（涉及协方差）；实践中常用启发式 $B(s) \approx \mathbb{E}[r | s]$ （即 value function）¹¹

对单参数模型，最优基线有闭式解：

$$B^*(s) = \frac{\mathbb{E}_a [\|\nabla_{\theta} \log \pi_{\theta}(a | s)\|^2 r(s, a)]}{\mathbb{E}_a [\|\nabla_{\theta} \log \pi_{\theta}(a | s)\|^2]}$$

- 多维情形下分子分母会变成协方差结构，计算昂贵。
- 实践中通常忽略梯度模长权重，退化为启发式： $B(s) \approx \mathbb{E}_a[r | s]$ ，即给定 prompt 的期望 reward（这就是 value function $V(s)$ 的定义）。
- 即使这个期望也算不出来，只能估计——但它给出了“基线应该长什么样”的指导原则。

¹¹ 视频画面时间区间：00:24:15 – 00:25:00。

4.5 优势函数

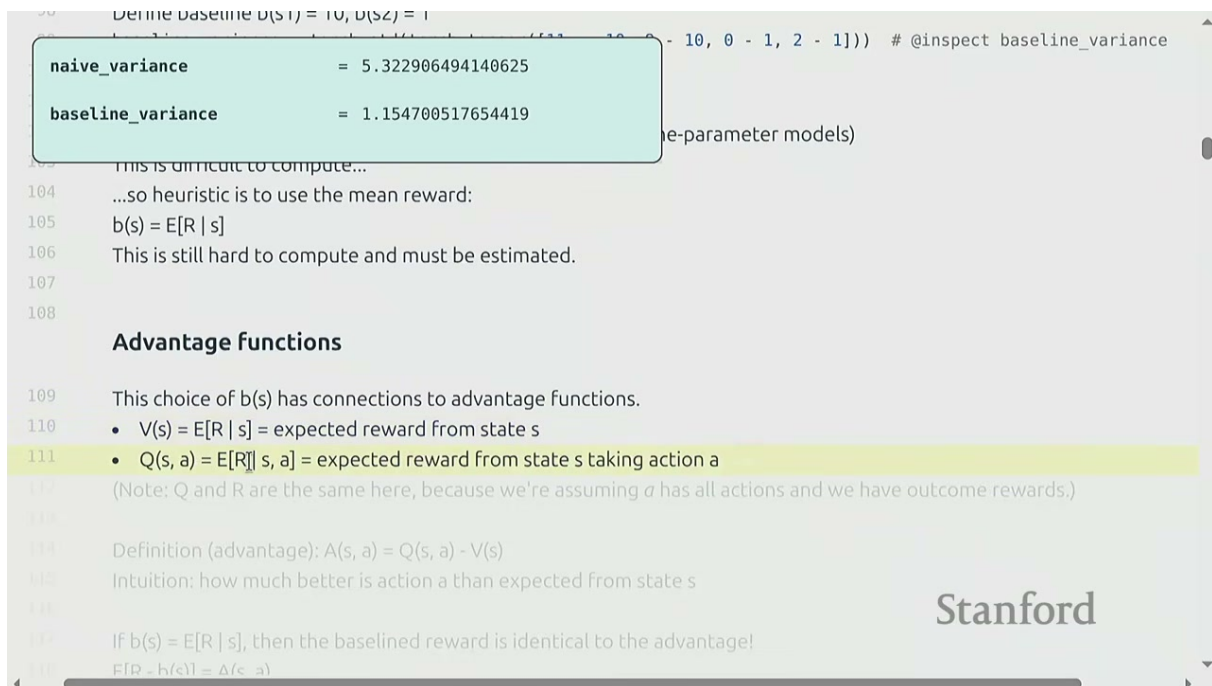


Figure 12: Value / Q / Advantage 的定义: $A(s, a) = Q(s, a) - V(s)$, 衡量动作 a 比平均水平好多少¹²

定义三个标准函数:

$$V(s) = \mathbb{E}_a[r | s], \quad Q(s, a) = r(s, a), \quad A(s, a) = Q(s, a) - V(s)$$

- $V(s)$: value function, 状态 s 下的期望 reward。
- $Q(s, a)$: 在结果奖励设定下, 因为 a 是整条响应, Q 就等于 $r(s, a)$ (一般 RL 里 Q 是回报的期望, 这里退化为单步 reward)。
- $A(s, a)$: advantage, 动作 a 相对平均水平的优劣。

基线即优势

若取启发式 $B(s) = V(s) = \mathbb{E}[r | s]$, 则 $r - B(s) = Q(s, a) - V(s) = A(s, a)$ 。即“reward 减基线”在这个选择下就是优势函数。这给出了基线的直观解释: 你不是在优化绝对 reward, 而是在优化“比平均水平好多少”。

¹²视频画面时间区间: 00:26:30 – 00:27:15。

4.6 统一的 delta 形式

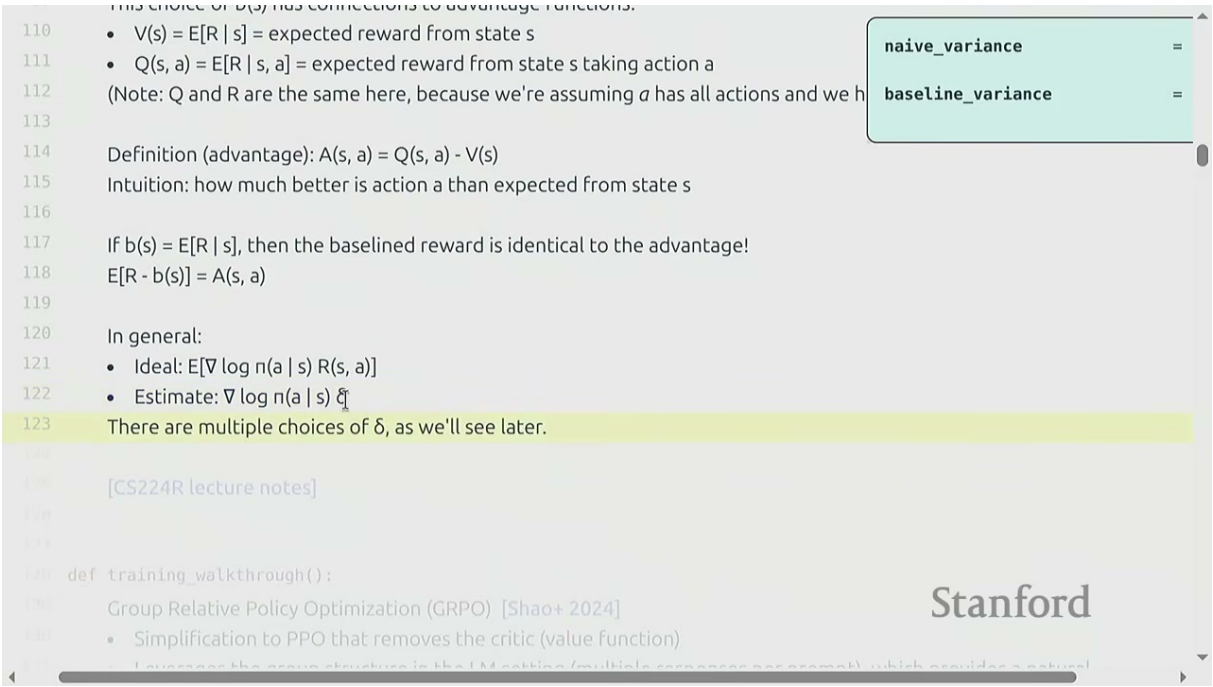


Figure 13: 所有策略梯度算法统一为 $\nabla \log \pi \cdot \delta$ ； δ 的不同选择对应不同算法（reward / 中心化 / 标准化）¹³

因为精确算不出这些期望，Percy 用 δ (delta) 统一记号——表示放进梯度的那个”东西”：

$$\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(a | s) \cdot \delta$$

δ 的选择	对应方法
$\delta = r$	朴素策略梯度
$\delta = r - \hat{V}(s)$	带基线（中心化 reward）
$\delta = (r - \hat{V}(s)) / \hat{\sigma}(s)$	GRPO 式（再除以标准差，尺度不变）

算法会变，框架不变

Percy 反复强调：“ δ 的具体形式不是重点——明年会有 GRPO2，但 $\nabla \log \pi \cdot \delta$ 这个框架是永恒的。”这也是为什么本讲花大力气建立这个统一视角。想看更完整的 RL 推导，他推荐 Chelsea Finn 的 Deep RL 课程。

¹³视频画面时间区间：00:28:30 – 00:29:15。

5 GRPO: 动机与伪代码

5.1 为什么 GRPO 在 2024 年才出现

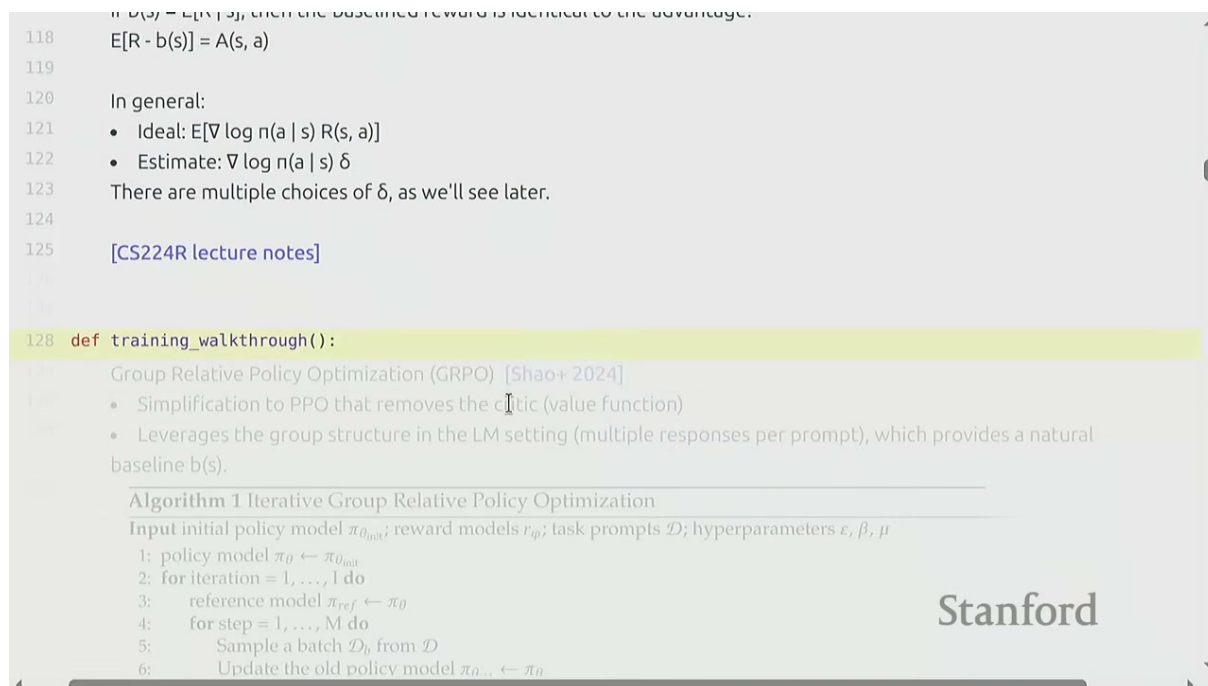


Figure 14: GRPO 的谱系: PPO → DPO → GRPO; GRPO 是 PPO 的简化, 依赖语言模型“一 prompt 多采样”的天然分组结构¹⁴

GRPO 的谱系是 PPO → DPO → GRPO。有趣的是, 算法往往是“从简单到复杂”演化, 但 GRPO 反而是 PPO 的简化。Percy 指出 GRPO 之所以到 2024 年 (PPO 是 2017 年) 才出现, 是因为语言模型设定提供了天然的分组结构:

GRPO 的核心洞察

对同一个 prompt 采样多条响应, 就得到一个自然组 (group)。组内 reward 的均值就是基线 $V(s)$ 的经验估计, 组内标准差给出归一化分母。组就是天然的对比集合——GRPO 的 advantage 是“相对组内同伴”的优劣。

经典 RL (机器人走路) 每个 rollout 长得都不一样, 没有自然分组, 只能用 value function 聚合所有可能状态-动作。语言模型的“一 prompt 多采样”让 GRPO 无需 value function。

¹⁴视频画面时间区间: 00:31:00 – 00:31:45。

5.2 GRPO 伪代码全貌

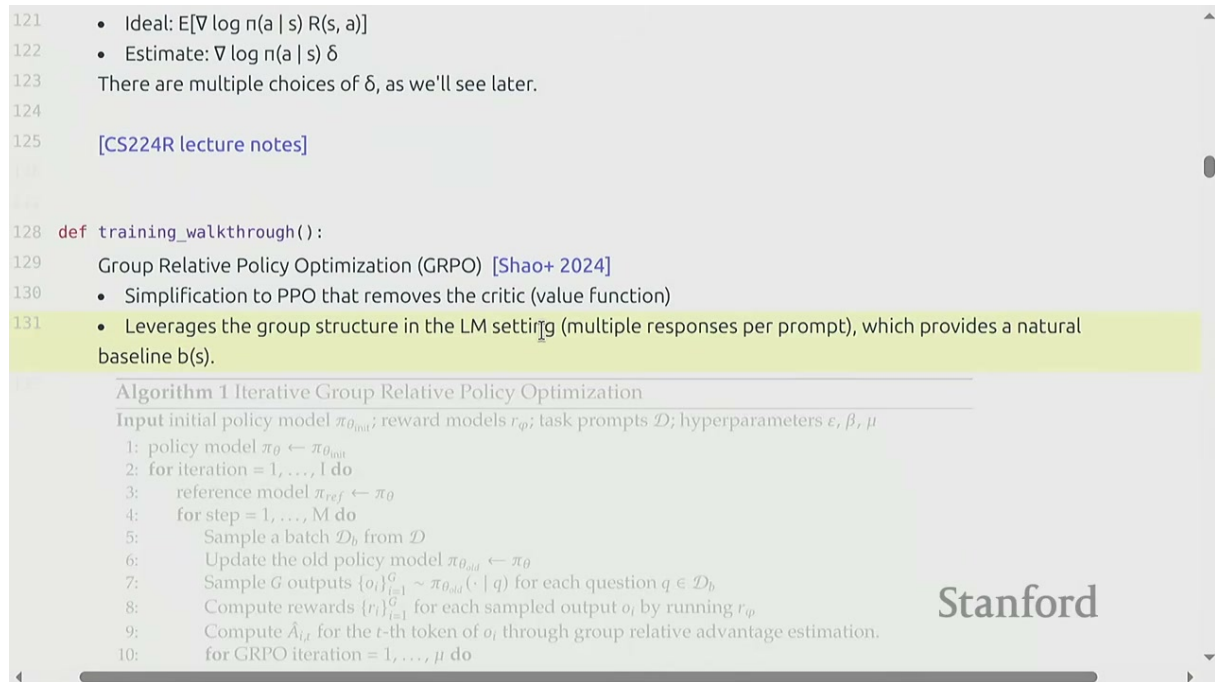


Figure 15: GRPO 伪代码全貌：rollout $\rightarrow$ reward $\rightarrow$ delta $\rightarrow$ log-prob $\rightarrow$ loss $\rightarrow$ 梯度步¹⁵

接下来 Percy 把伪代码逐块翻译成可运行 Python。整个流程概括为：

1. 生成响应：给定固定模型，对每个 prompt 采样 G 条响应。
2. 算 reward：对每条响应调用奖励函数（与模型无关，除非用 LM-as-judge）。
3. 算 delta：根据所选 δ 变体（原始/中心化/标准化）转换 reward。
4. 算 log-prob：把响应过模型，取出每个 token 的对数概率。
5. 算 loss：用 log-prob 和 delta 组合（clip 或不 clip），可加 KL 惩罚。
6. 走梯度步。

¹⁵视频画面时间区间：00:32:30 – 00:33:15。

6 玩具任务：排序与奖励设计

6.1 为什么用排序任务

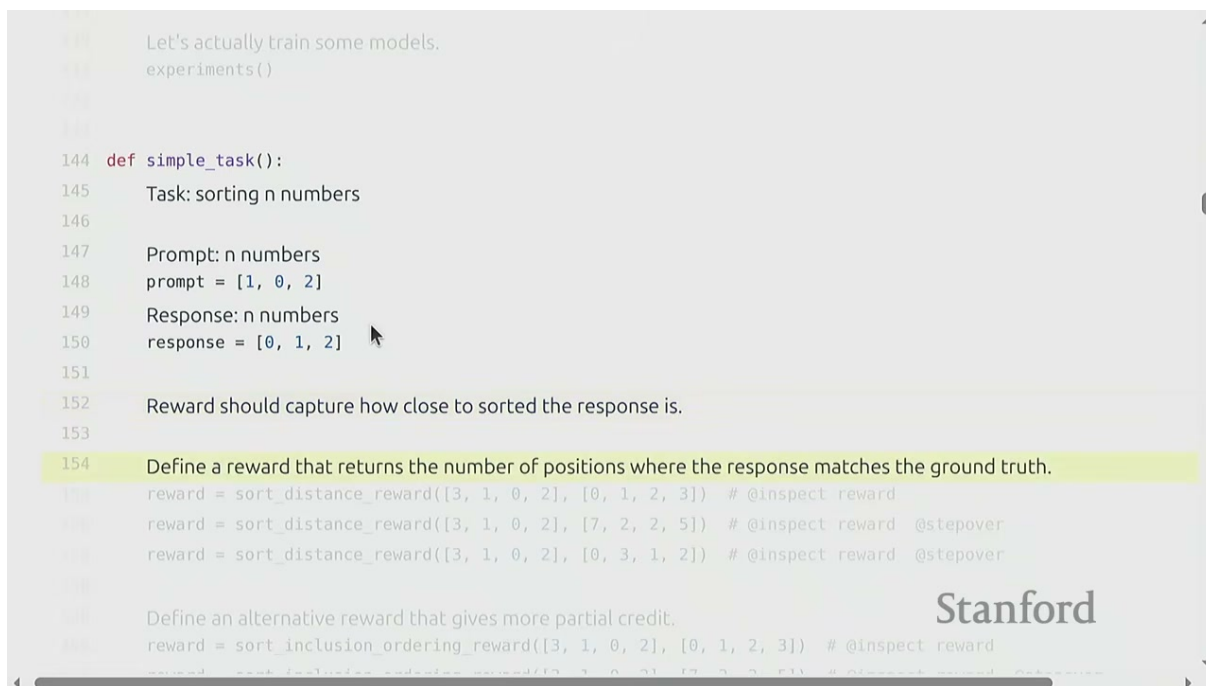


Figure 16: 玩具任务：排序 n 个数。Percy 用它是因为可以在笔记本上用极简模型训练，无需 Transformer¹⁶

为了能在没有 GPU 的笔记本上跑通，Percy 设计了一个极简任务：排序 n 个数。prompt 是 n 个数，响应是希望排好序的 n 个数。

奖励设计是 RL 的艺术

最直接的奖励是“完全排好序给 1 否则给 0”，但这会触发稀疏 reward 陷阱——策略太差时全得 0，无法更新。因此 Percy 设计了两种给部分信用（partial credit）的奖励。

¹⁶视频画面时间区间：00:34:15 – 00:35:00。

6.2 两种奖励函数与部分信用

```
def sort_distance_reward(prompt: list[int], response: list[int]) -> float: # @inspect prompt, @inspect response
    """
    Return how close response is to ground_truth = sorted(prompt).
    In particular, compute number of positions where response[i] == sorted(prompt)[i].
    """
    assert len(prompt) == len(response)
    ground_truth = sorted(prompt)
    return sum(1 for x, y in zip(response, ground_truth) if x == y)

def sort_inclusion_ordering_reward(prompt: list[int], response: list[int]) -> float: # @inspect prompt, @inspect response
    """
    Return how close response is to ground_truth = sorted(prompt).
    In particular, compute number of positions where response[i] == sorted(prompt)[i].
    """
    assert len(prompt) == len(response)

    # Give one point for each token in the prompt that shows up in the response
    inclusion_reward = sum(1 for x in prompt if x in response) # @inspect inclusion_reward

    # Give one point for each adjacent pair in response that's sorted
    ordering_reward = sum(1 for x, y in zip(response, response[1:]) if x <= y) # @inspect ordering_reward

    return inclusion_reward + ordering_reward
```

Callout box values:

prompt	=	[3, 1, 0, 2]
response	=	[0, 1, 2, 3]
inclusion_reward	=	4

Figure 17: 两种部分信用奖励：按位置匹配数；按 inclusion + 相邻对有序数。注意第二种有“漏洞”¹⁷

奖励 1（位置匹配）：响应与 ground truth（即 prompt 的排序）逐位置比对，数匹配的位置数。

$$r_1 = \sum_{i=1}^n \mathbb{I}[\text{resp}_i = \text{sorted}(\text{prompt})_i]$$

奖励 2（inclusion + 相邻有序）：每个出现在响应里的 prompt token 得 1 分（鼓励至少输出对的 token）；每个相邻对有序再得 1 分。两者相加。

$$r_2 = \underbrace{\sum_{t \in \text{prompt}} \mathbb{I}[t \in \text{resp}]}_{\text{inclusion}} + \underbrace{\sum_{i=1}^{n-1} \mathbb{I}[\text{resp}_i \leq \text{resp}_{i+1}]}_{\text{adjacent sorted pairs}}$$

找漏洞（思考题）

奖励 2 有一个漏洞——Percy 留给学生思考。提示：inclusion 只要求 token 出现，不要求每个只出现一次；相邻有序也不要求用尽所有 token。你能构造一个 reward 很高但其实没真正排序的响应吗？（比如重复输出最小的那个数。）这正是奖励设计的典型陷阱：部分信用太宽松，模型会钻空子去摘低垂的果实。

本讲后续统一采用奖励 2。值得注意的是，即便很离谱的响应也能拿到不低的分数（比如努力“试图”排序的给 6 分），这为早期低能策略提供了梯度信号——正是对抗稀疏 reward 的关键。

¹⁷ 视频画面时间区间：00:36:15 – 00:37:00。

7 极简模型与推理

7.1 模型架构

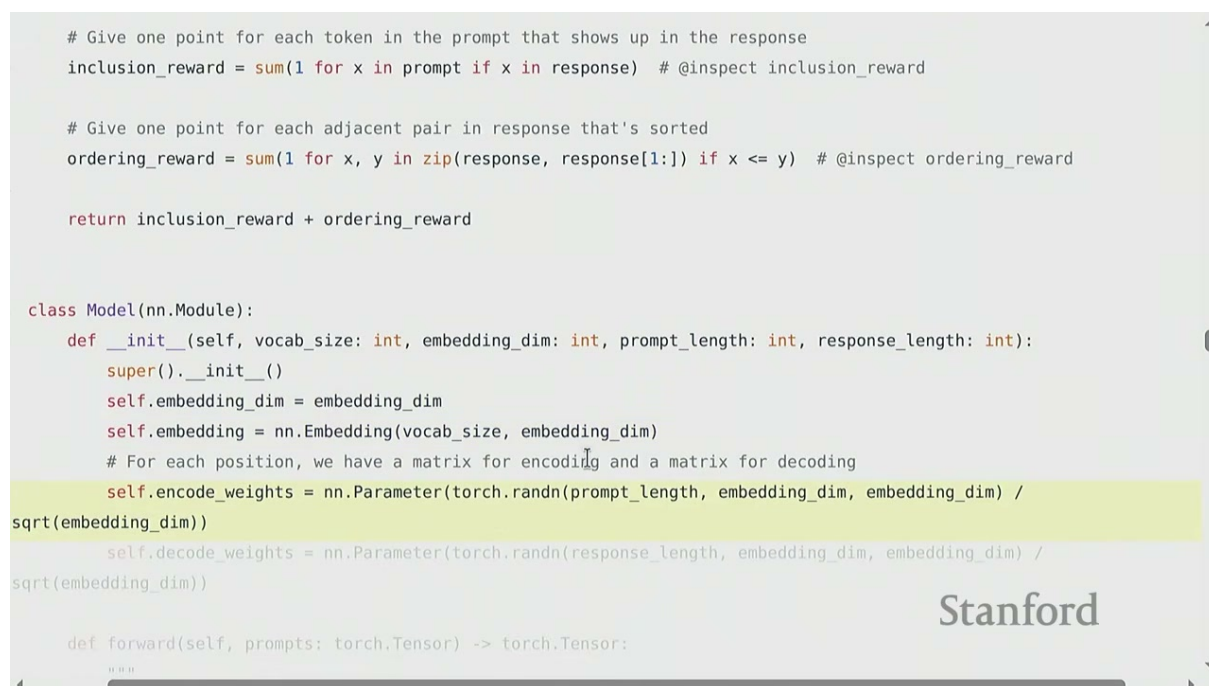


Figure 18: 极简模型：每位置独立的 encode/decode 矩阵；非自回归——各位置独立解码¹⁸

为了让代码干净，Percy 设计了一个非自回归的极简模型（真实 LM 是自回归的，这里简化）：

- 假设 prompt 和响应长度固定且相等（都为 n ）。
- 用每位置独立的参数捕获位置信息（排序里位置是核心）。
- 独立解码每个位置：先编码整个序列，再对每个位置输出 token 分布（不是逐 token 自回归生成）。

为什么要偏离自回归？

Percy 坦言：自回归生成那段代码“非常 hairy”，为了让讲解清晰他做了简化。这种“独立解码”在真实场景只在 speculative decoding 等少数地方有限使用，一般不这么做。但作为教学载体，它把注意力引向 RL 机制本身而非生成工程。

模型组件：一个 embedding 矩阵；每个位置一个 encode 矩阵 ($d_1 \rightarrow d_2$) 和一个 decode 矩阵；最后映射到 vocab logits。

¹⁸视频画面时间区间：00:38:30 – 00:39:15。

7.2 前向与采样

```
self.embedding = nn.Embedding(vocab_size, embedding_dim)
# For each position, we have a matrix for encoding and a matrix for decoding
self.encode_weights = nn.Parameter(torch.randn(prompt_length, embedding_dim, embedding_dim) /
sqrt(embedding_dim))
self.decode_weights = nn.Parameter(torch.randn(response_length, embedding_dim, embedding_dim) /
sqrt(embedding_dim))

def forward(self, prompts: torch.Tensor) -> torch.Tensor:
    """
    Args:
        prompts: int[batch pos]
    Returns:
        logits: float[batch pos vocab]
    """
    # Embed the prompts
    embeddings = self.embedding(prompts) # [batch pos dim]

    # Transform using per prompt position matrix, collapse into one vector
    encoded = einsum(embeddings, self.encode_weights, "batch pos dim1, pos dim1 dim2 -> batch dim2")

    # Turn into one vector per response position
    decoded = einsum(encoded, self.decode_weights, "batch dim2, pos dim2 dim1 -> batch pos dim1")
```

Figure 19: 推理流程：embed prompt → 逐位置变换 → logits → 采样 G 条响应；维度 (batch, pos, trial)¹⁹

推理（生成 rollout）的维度流：

1. prompt 形状 (B, n) ， B 为 batch（prompt 数）。
2. embed 到 (B, n, d_1) ，经每位置 encode 矩阵并按位置求和塌缩到 (B, d_2) 。
3. 再对每位置用 decode 矩阵回到 (B, n, d_2) ，最后映到 logits (B, n, V) ， V 为 vocab。
4. 把 logits 展平，对每 (batch, pos) 采样 G 条 → 响应 (B, n, G) ，重排为 (B, G, n) 。

$$\text{logits} : (B, n, V) \xrightarrow{\text{sample } G} \text{resp} : (B, G, n)$$

为什么响应“很接近”

因为同一 (batch, pos) 的 G 条响应都从同一 logits 分布独立采样，自然相似。这和真实 LM 从同分布多次采样同理，唯一区别是真实 LM 自回归、本模型独立。Percy 本想改温度引入更多 reward 方差，但为忠实于算法保持原样。

¹⁹ 视频画面时间区间：00:40:00 – 00:40:45。

8 Delta、Log-prob 与损失

8.1 计算 delta：算法菜单

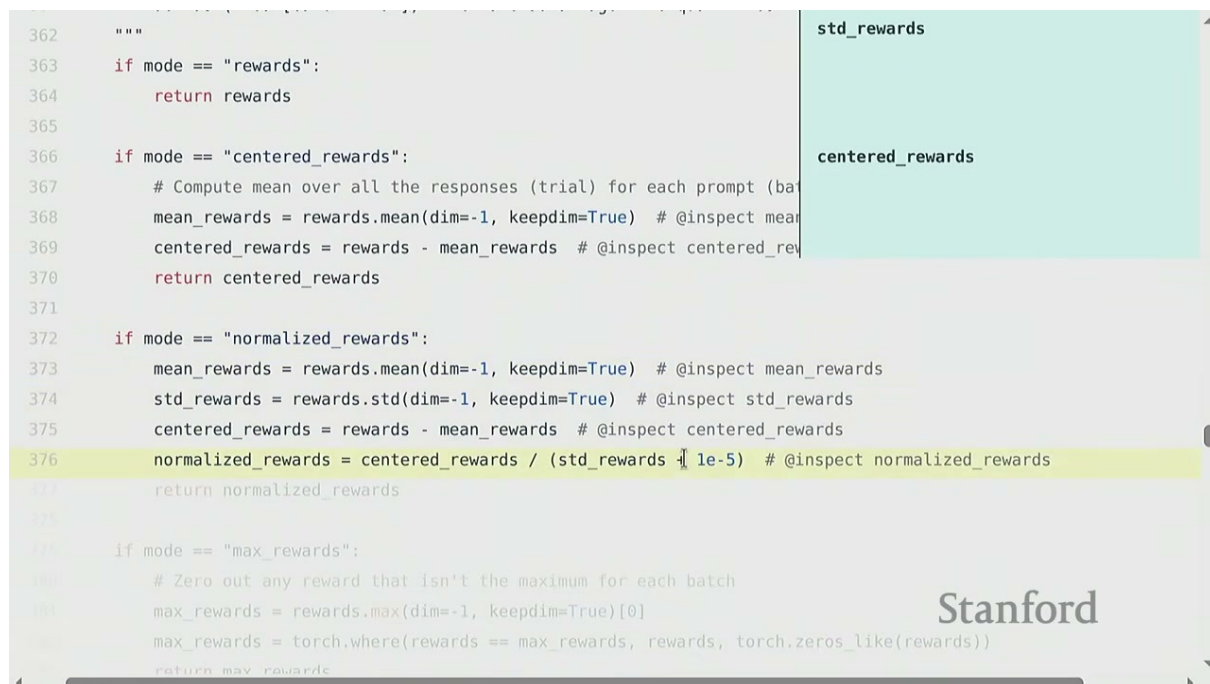


Figure 20: compute_deltas 函数：原始 reward / 中心化（减均值）/ GRPO 标准化（再除标准差）/ 只保留最大值²⁰

compute_deltas(rewards) 把 reward 转成 δ 。Percy 把它呈现为一份“菜单”，对应 Assignment 5 学生要探索的选择：

1. 原始： $\delta = r$ （朴素策略梯度）。
2. 中心化（centered）：每个 prompt 组内减均值。

$$\delta_{i,g} = r_{i,g} - \frac{1}{G} \sum_{g'} r_{i,g'}$$

回答了之前的提问：若 reward 是 $\{1, 0, 0, \dots\}$ ，均值约 0.1，减去后错答得到负 δ ，于是也会被“推远”。

3. GRPO 标准化（normalized）：中心化后再除标准差（加小 ϵ 防除零）。

$$\delta_{i,g} = \frac{r_{i,g} - \bar{r}_i}{\sigma_i + \epsilon}$$

4. 只保留最大（bonus）：把非组内最大的 reward 置零——“全有或全无”，对抗部分信用导致的局部最优。

两个关键性质

- 全相同则不更新：若组内 reward 全是 5，中心化后 δ 全 0，不更新。直觉：组内没有相对差异，无法判断该偏爱谁，于是弃权，等别的样本给信号。
- 尺度不变：标准化后 reward 整体乘 100 不改变 δ ——对奖励的绝对尺度不敏感，这是 GRPO 的优良性质。

²⁰ 视频画面时间区间：00:45:15 – 00:46:00。

8.2 计算 log-prob

```
334 def compute_log_probs(prompts: torch.Tensor, responses: torch.Tensor, model: Model) -> torch.Tensor:
335     """
336     Args:
337         prompts (int[batch pos])
338         responses (int[batch trial pos])
339     Returns:
340         log_probs (float[batch trial pos]) under the model
341     """
342     # Compute log prob of responses under model
343     logits = model(prompts) # [batch pos vocab]
344     log_probs = F.log_softmax(logits, dim=-1) # [batch pos vocab]
345
346     # Replicate to align with responses
347     num_responses = responses.shape[1]
348     log_probs = repeat(log_probs, "batch pos vocab -> batch trial pos vocab", trial=num_responses) # [batch trial
vocab]
349
350     # Index into log probs using responses
351     log_probs = log_probs.gather(dim=-1, index=responses.unsqueeze(-1)).squeeze(-1) # [batch trial pos]
```

Figure 21: log-prob 计算：把响应过模型取 logits $\rightarrow$ log-softmax $\rightarrow$ 按响应 token 索引取出；形状 (batch, trial, pos)²¹

策略梯度还需要 $\nabla \log \pi$ ，所以要先拿到每条响应的对数概率：

1. 响应过模型 $\rightarrow$ logits (B, G, n, V) 。
2. log-softmax 得到 log-prob。
3. 用响应张量作为索引，从 log-prob 中 gather 出每个实际 token 的对数概率 $\rightarrow (B, G, n)$ 。

$$\log \pi_{\theta}(\text{resp}) = \sum_{t=1}^n \log p_{\theta}(\text{resp}_t \mid \dots)$$

因为 outcome reward 的 δ 不依赖位置，它会被广播到每个位置；若用 process reward， δ 也会逐位置。

²¹ 视频画面时间区间：00:48:15 – 00:49:00。

8.3 no_grad 插曲：冻结的艺术

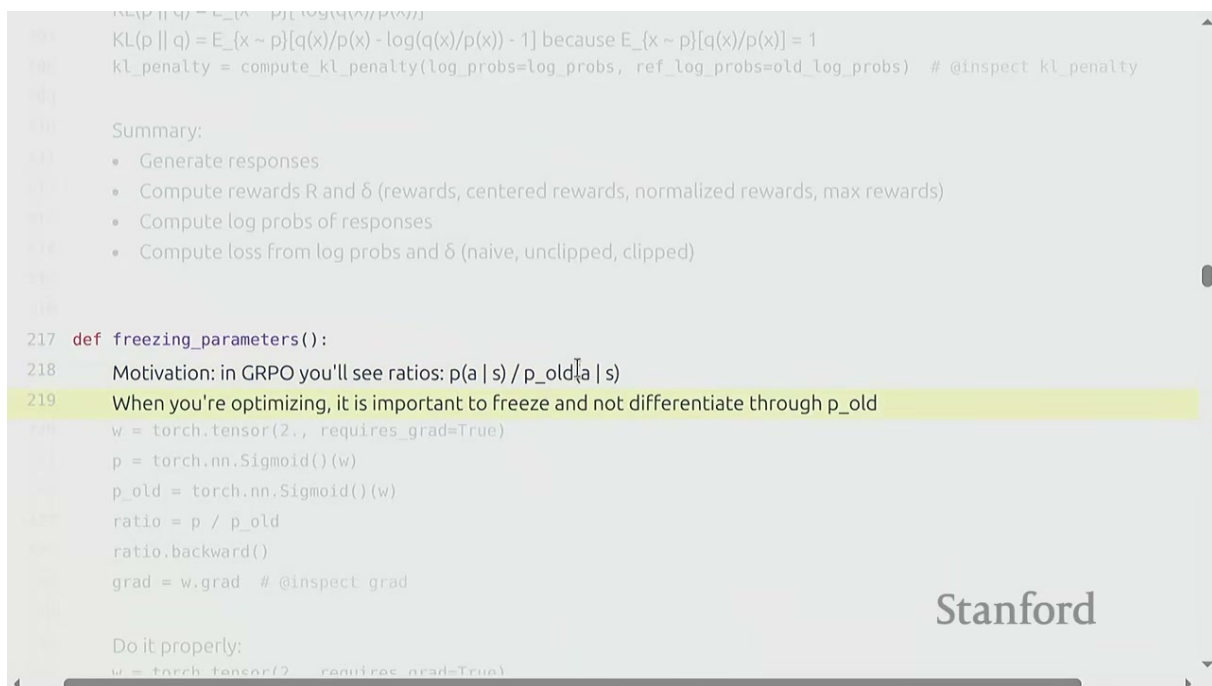


Figure 22: no_grad 插曲：除以 p_{old} 时若不冻结，比值恒为 1、梯度为 0；必须用 `torch.no_grad()` 隔离计算图²²

PPO/GRPO 里到处是“概率比” $\pi_{\theta}/\pi_{\text{old}}$ 。这里有个致命陷阱。设单参数 w 同时驱动 p 和 p_{old} ：

$$\text{ratio} = \frac{p(w)}{p_{\text{old}}(w)} = 1 \implies \nabla_w 1 = 0$$

常见 bug：梯度全为零

若 p_{old} 也参与计算图，比值求导恒为零，模型完全不学。必须用 `torch.no_grad()` 把任何要当常数的量包起来——拿到数值但切断梯度。RL 与预训练的关键区别：RL 里同时存在“新参数”和“旧/冻结参数”，要时刻清楚谁在计算图里。

²² 视频画面时间区间：00:50:45 – 00:51:30。

8.4 GRPO 的 clipped loss

```

187 deltas = compute_deltas(rewards=rewards, mode="centered_reward")
188 deltas = compute_deltas(rewards=rewards, mode="normalized_reward")
189 deltas = compute_deltas(rewards=rewards, mode="max_rewards")
190
191 Compute log probabilities of these responses:
192 log_probs = compute_log_probs(prompts=prompts, responses=responses)
193
194 Compute loss so that we can use to update the model parameters
195 loss = compute_loss(log_probs=log_probs, deltas=deltas, mode="naive") # @inspect loss
196
197 freezing_parameters()
198
199 old_model = Model(vocab_size=3, embedding_dim=10, prompt_length=3, response_length=3) # Pretend this is a
checkpoint @stepover
200 old_log_probs = compute_log_probs(prompts=prompts, responses=responses, model=old_model) # @stepover
201 loss = compute_loss(log_probs=log_probs, deltas=deltas, mode="unclipped", old_log_probs=old_log_probs) #
loss
202 loss = compute_loss(log_probs=log_probs, deltas=deltas, mode="clipped", old_log_probs=old_log_probs) # @i
203
204 Sometimes, we can use an explicit KL penalty to regularize the model.
205 KL(p || q) = E_{x ~ p}[log(p(x)/q(x))]
206 D(f || g) = \int f(x) \log(f(x)/g(x)) dx

```

Figure 23: GRPO 损失: $\text{ratio} \cdot \delta$ 与 $\text{clip}(\text{ratio}) \cdot \delta$ 取 min, 再加负号变 loss; clip 把更新幅度限制在 $[1 - \epsilon, 1 + \epsilon]$ ²³

GRPO 损失 (clip 版) :

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E} [\min(\rho \delta, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) \delta)]$$

其中 $\rho = \pi_{\theta}(a | s) / \pi_{\text{old}}(a | s) = \exp(\log \pi_{\theta} - \log \pi_{\text{old}})$ 。

clip 的作用

- 若 $\text{ratio } \rho$ 落在 $[1 - \epsilon, 1 + \epsilon]$ 内, clip 不生效, 等价于朴素策略梯度 (只是多除了个常数 p_{old} , 相当于缩放梯度)。
- 若 ρ 超出范围, 更新幅度被夹在 $[1 - \epsilon, 1 + \epsilon]$ 乘 δ 。目的: 防止策略在单步内偏离 old 策略太远, 保持训练稳定。
- 取 min 是 PPO 的标准技巧 (trust region 思想); 外层加负号是因为把“最大化 reward”翻成“最小化 loss”。

²³ 视频画面时间区间: 00:53:45 – 00:54:30。

8.5 KL 惩罚：低方差估计

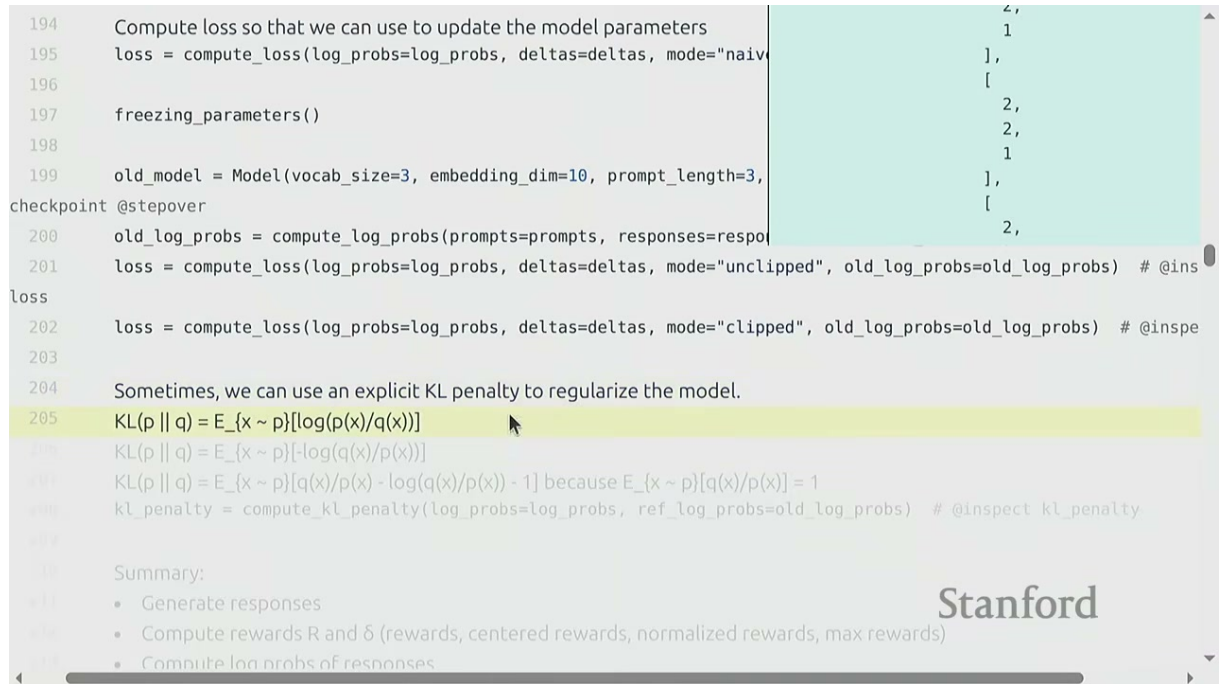


Figure 24: KL 惩罚推导： $D_{KL}(q||p) = \mathbb{E}_q[\log(q/p)]$ ；用 $q/p - \log(q/p) - 1$ 得到同期望但更低方差的无偏估计²⁴

GRPO 常加一个 KL 正则，把策略拉向参考模型 π_{ref} （防漂移/防 reward hacking）。KL 散度定义：

$$D_{KL}(q || p) = \mathbb{E}_{x \sim q} \left[\log \frac{q(x)}{p(x)} \right]$$

朴素无偏估计是直接采样求 $\log(q/p)$ 的均值。但有个更低方差的等价形式：

$$\hat{D}_{KL} = \mathbb{E}_{x \sim q} \left[\frac{q(x)}{p(x)} - \log \frac{q(x)}{p(x)} - 1 \right]$$

为什么等价？

因为 $\mathbb{E}_{x \sim q}[q(x)/p(x)] = \int q \cdot \frac{q}{p} dx \dots$ 这里的关键是 $\mathbb{E}_q[q/p]$ 这一项按重写后等于常数 1（在理想情况下），减 1 抵消，剩下的就是原始 KL。而把 q/p 这一项塞进来降低了估计方差。这是 RL 的常见手艺：把估计式重写成期望相同但方差更低的形式（控制变量/control variate 的思想）。

实现上，KL 惩罚在 vocab 维度求和、在 (batch, trial, pos) 维度取均值。Percy 说实际中加不加“区别不大”，是个可选的正则项。

²⁴视频画面时间区间：00:56:15 – 00:57:00。

9 完整训练循环

9.1 三层循环结构

```
447     optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
448
449     records = []
450     ref_log_probs = None
451     ref_model = None
452     old_log_probs = None
453
454     if use_cache:
455         out = open(log_path, "w")
456     else:
457         out = sys.stdout
458
459     for epoch in tqdm(range(num_epochs), desc="epoch"):
460         # If using KL penalty, need to get the reference model (freeze it every few epochs)
461         if kl_penalty != 0:
462             if epoch % compute_ref_model_period == 0:
463                 ref_model = model.clone()
464
465         # Sample responses and evaluate their rewards
466         responses = generate_responses(prompts=prompts, model=model, num_responses=num_responses)
467         rewards = compute_reward(prompts=prompts, responses=responses, reward_fn=reward_fn) # [batch trial]
468         deltas = compute_deltas(rewards=rewards, model=model) # [batch trial]
```

Figure 25: 完整算法：外层 epoch 循环（更新 reference）→ 采样响应 → 内层多步梯度更新；推理昂贵，故对同一批响应做多次更新²⁵

把所有组件拼起来，GRPO 训练循环的骨架：

```
1 model = SimpleModel(...); opt = AdamW(model.parameters())
2 ref_model = copy(model) # reference / anchor, updated slower
3 for epoch in range(E):
4     # if using KL penalty, freeze a reference copy first
5     ref_model = maybe_update_ref(model, ref_model)
6     for prompts in dataloader:
7         # 1) rollout generation (inference, expensive)
8         responses = model.generate(prompts, num_responses=G)
9         # 2) reward + delta (model-independent)
10        rewards = reward_fn(prompts, responses)
11        deltas = compute_deltas(rewards) # raw/centered/normalized
12        # 3) old policy log-prob (frozen, for ratio)
13        with torch.no_grad():
14            old_logp = model.log_prob(responses)
15        # 4) inner loop: multiple updates on same responses
16        for step in range(inner_steps):
17            logp = model.log_prob(responses)
18            loss = grpo_loss(logp, old_logp, deltas, eps) \
19                    + beta * kl_penalty(model, ref_model, responses)
20            opt.step(loss.backward())
```

为什么有内层循环？

推理（rollout）非常昂贵，所以拿到一批响应后要多走几个梯度步榨干它的价值。这就是双层循环的动机：外层重新采样，内层多次更新。若加 KL，还多一个最外层更新参考模型的循环（reference 比 policy 更新更慢）。

²⁵ 视频画面时间区间：01:00:15 – 01:01:00。

9.2 参考模型与“慢节奏”

```
455     out = open(log_path, "w")
456 else:
457     out = sys.stdout
458
459 for epoch in tqdm(range(num_epochs), desc="epoch"):
460     # If using KL penalty, need to get the reference model (freeze it every few epochs)
461     if kl_penalty != 0:
462         if epoch % compute_ref_model_period == 0:
463             ref_model = model.clone()
464
465     # Sample responses and evaluate their rewards
466     responses = generate_responses(prompts=prompts, model=model, num_responses=num_responses) # [batch trial p
467     rewards = compute_reward(prompts=prompts, responses=responses, reward_fn=reward_fn) # [batch trial]
468     deltas = compute_deltas(rewards=rewards, mode=deltas_mode) # [batch trial]
469
470     if kl_penalty != 0: # Compute under the reference model
471         with torch.no_grad():
472             ref_log_probs = compute_log_probs(prompts=prompts, responses=responses, model=ref_model) # [batch 1
473
474     if loss_mode != "naive": # Compute under the current model
475         with torch.no_grad():
476             old_log_probs = compute_log_probs(prompts=prompts, responses=responses, model=model) # [batch trial]
```

Figure 26: 大局：外层生成 rollout + 内层多次更新；KL 惩罚则引入比策略更新更慢的参考模型作为锚点²⁶

三层“时间尺度”

GRPO 的完整图景包含三个不同节奏：

- 响应采样（最慢、最贵）：固定模型，采样一批 rollout。
- 策略更新（中速）：对这批 rollout 走 inner_steps 步梯度。
- 参考模型（最慢更新）：KL 惩罚锚定的 π_{ref} ，更新频率比策略还低。

Percy 的代码版为了简洁把 reference 更新得“有点频繁”，与官方 GRPO 实现的“三循环”略有出入，但教学上已足够。

10 Assignment 5 与延伸

10.1 与作业的对应

本讲整个“算法菜单”正是 Assignment 5 的探索空间。学生要在真实（或半真实）的 LM 上实现 RL，并比较不同 δ 变体、clip 与否、KL 与否的效果。

²⁶ 视频画面时间区间：01:02:30 – 01:03:15。

A5 的关键决策点

- δ 选择：原始 / 中心化 / GRPO 标准化 / 只保留最大——不同选择影响收敛速度与最终性能。
- clip：开/关， ϵ 取值——稳定性与学习速度的权衡。
- KL 惩罚：开/关， β 取值——是否需要锚定参考模型。
- 奖励设计：稀疏 0/1 还是部分信用——直接决定能否逃出冷启动陷阱。
- 工程细节：no_grad 包裹旧/参考量、gather log-prob、维度对齐。

10.2 过程奖励与多轮交互

Percy 在问答中补充了两个方向：

- 过程奖励 (process reward)：outcome reward 只在结尾给信号；若能在中间步骤评估（如每步推理是否正确），梯度会更稠密、方差更低。但 LM 的过程奖励极难做好——除非明显跑偏，否则很难判断中间步骤的好坏。
- 尽早评估：如果有可信的中间信号，应尽早评估 reward。

关于折扣 (discounting)：有人问“是否该给结尾 token 更大权重”。Percy 认为不明确——早期决策往往更重要（选对策略后续自然展开），而 RL 的难处正在于不知道信用该分配给谁，所以这里把信用“均匀涂抹”在整条轨迹上。

10.3 本讲小结

核心要点回顾

1. 设置：语言模型 RL 的状态是 token 序列，动作是下一 token，转移动力学是简单拼接——这使规划 (test-time compute) 成为可能，状态可被模型“自由构造”。
2. 策略梯度： $\nabla J \approx \nabla \log \pi \cdot \delta$ 是永恒框架； δ 的具体形式 (reward/中心化/标准化) 对应不同算法。
3. 基线与优势：减去只依赖状态的基线不改变期望梯度但降方差；选 $B(s) = V(s)$ 时 advantage 凸显“比平均好多少”。
4. GRPO：利用“一 prompt 多采样”的天然分组估计 $V(s)$ 和 $\sigma(s)$ ，免去 value function；clip 稳定训练，KL 惩罚锚定参考模型。
5. 工程纪律：旧/参考量必须 no_grad；推理昂贵故用内层多步更新；reward 设计要给部分信用以对抗稀疏陷阱。

10.4 配套阅读

材料	对应知识点
PPO: Schulman et al. 2017	clip ratio、trust region
GRPO: DeepSeek 2024	组内标准化、去 value function
Chelsea Finn: CS285 Deep RL	策略梯度与基线的完整推导
Assignment 5 (CS336)	本讲代码的实战实现